

ClearCare+ Complete Documentation

PROGRAM: Management – Digitalization and Transformation, M.A

COURSE: Change Process Management

MEMBERS:

Pablo Perez Castro Frade: Overall team alignment, business process analyst and architect, prototype iteration support, supporting documentations

Devashri Trivedi: Value proposition, customer journey mapping, main prototype iteration, supporting database researcher, supporting documentations

Karim Elkhodery: Customer journey mapping, prototype iteration support, primary API, AI, and database researcher, supporting documentations

Darshan Shanmugasundaram: Supporting database researcher, supporting documentations

Patricia Ysabel Casupang: App developer and UI/UX localizer, primary technical engineer, translated the architecture into the tech stack, primary documentations

MDT 25-27 | Change and Process Management Project. AI-first health-insurance doctor-booking SaaS for immigrants in Germany.

GitHub Repository: github.com/pcsphsb/clearcare-plus

README: github.com/pcsphsb/clearcare-plus/blob/main/README.md

Live: pcsphsb.github.io/clearcare-plus

n8n: [ClearCare+ Complete Workflow](#)

The n8n link points to the team's private n8n Cloud workspace, so it opens only for an account with access. To open and run the workflow in your own account, import the exported file Audit/Documentation/WorkfLow JSON SQL/ClearCare+ Complete WorkfLow.json into your own n8n (Workflows, then Import from File). See Part D.

Table of Contents

Summary at a Glance

Part A. Business and Process Concept

1. Problem
2. Benchmark: TK-Doc (Techniker Krankenkasse)
3. Solution: ClearCare+
4. AI at Every Step (designed)
5. Customer Journey
6. Process Architecture (APQC-style)
7. BPMN 2.0: Designed vs Delivered
8. Regulatory, Compliance, and Governance
9. Interface (GUI/VUI), Inputs, and Outputs

Part B. System Architecture and Build 10. What It Is and the Stack 11. Architecture 12. Run Modes 13. Pages and Flows 14. Data Model and Security (RLS) 15. Feature Areas 16. Internationalization and UI Frame 17. Build Log 18. File Map

Part C. Edge Functions (Standalone Engine)

Part D. n8n Workflow (Demo Engine) 19. Why n8n, and the Auth Boundary 20. The One Consolidated Workflow 21. Cleanup Performed 22. Registration Email (FLOW 4) 23. NULL-Field Fixes 24. Import, Activate, Present 25. AI Triage System Prompt 26. n8n Workflow Files

Part E. Doctor Data 27. Source and Import 28. Provider Adapter Pattern

Part F. ArztAPI Scaffolding

Part G. Future Sprints

AI Assistance Disclosure

AI Assistance was used in order to work twice faster given the current project timeline and limited manpower to deploy a full tech-stack application. The student's learning of understanding the process is prioritized and AI was primarily used as a second engineer if completing the construction of the codebase.

- Claude Pro (Opus 4.8) was used for parallel development of the application code.
 - See Commits History for Co-Authored with Claude.
 - Most prominently used in post foundational construction (For Email Configuration setup and styling through Supabase Templates).
- Business Process Modelling, Workflow Modelling, Decision-Making, Tech Architecture and End-to-End Process Mapping all directed and constructed by the students.

This file consolidates all eight project documents into one compressed reference. The eight sources are the Business/SaaS concept, System Overview, n8n Workflow, n8n Consolidation, Edge Functions, Doctor File Import, Doctor Data Architecture, and ArztAPI Scaffolding. The README and the per-topic docs in `Workflow Documentation/` remain the long-form sources.

Summary at a Glance

Product	AI-first health-navigation SaaS for immigrants in Germany
Core user	Non-German-speaking immigrants (GKV/PKV)
Problem solved	End-to-end failure from insurance confusion to the phone-call booking wall
Differentiator	Single connected flow and automated booking vs TK's separate modules and human phone call
AI coverage	16 BPMN elements: 9 live, 4 partial, 3 future
Delivered build	Static front-end (6 langs) + Supabase (Auth/Postgres/RLS) + Groq triage + OSM fallback; two engines via USE_N8N
Business model	B2B2C (white-label to insurers) or D2C subscription
Benchmark	TK-Doc (fragmented, German-first, manual booking)
Regulatory	GDPR (Art. 9 and 22) and EU AI Act; burden concentrated in triage
Compliance strategy	"Navigation, not diagnosis" plus human-in-the-loop plus compliance-by-design

PART A. BUSINESS AND PROCESS CONCEPT

1. Problem

A non-German-speaking immigrant who needs care faces a chain of process failures, not just a language barrier. There is **insurance confusion** (GKV vs PKV, with German-only rules). There is **referral dependency** (specialists need a GP *Überweisung* first). There is the problem of **finding the right doctor** (language, insurance, and availability all vary). And there is the **booking wall** (booking is a German phone call to a receptionist). The result is that people delay care, misnavigate, or give up. *Student experience*: even with English insurance support, phone middlemen cause waits and errors. Direct, digitized, automated booking removes the manual phone-call bottleneck.

2. Benchmark: TK-Doc (Techniker Krankenkasse)

TK-Doc has a Symptom Checker, a Doctor Locator, Video Consultation, and Appointment Booking. But these are **separate modules**. The app is German-first. Booking is still done by a human phone agent. The **critical gap** is that it is not a connected end-to-end process. The phone-call booking step is exactly where non-German speakers drop off.

3. Solution: ClearCare+

ClearCare+ is an **AI-first health-navigation SaaS**. It takes a user from "*I don't feel well*" to "*appointment confirmed*" in their own language, with no phone call and no manual search. Differentiators versus TK: a single connected flow, AI-automated booking that the user confirms, a language-first design, live referral guidance, and live reschedule/cancel. The AI insurance explainer and post-appointment features are future scope.

Core value proposition: For newly arrived immigrants, ClearCare+ replaces the confusing German-language find-and-book process with one AI-guided experience in the user's language. There are three value layers. **Language:** the whole app and the AI replies use the user's chosen language, no matter what language the user types in. **Process:** triage, matching, booking, and follow-up are connected.

Understanding: a future AI policy explainer. A referral footnote already exists in the triage UI.

4. AI at Every Step (designed)

Onboarding is a manual form that auto-feeds later steps. Symptom triage gives an AI care-level recommendation. Referral navigation flags when a referral is needed. Doctor matching uses AI on language, insurance, specialty, location, and availability. Booking is AI-automated. Confirmation and prep follow. Post-appointment follow-up is future scope.

5. Customer Journey

TRIGGER, ONBOARDING, UNDERSTAND, TRIAGE, MATCH, BOOK, ATTEND, FOLLOW-UP. One-time onboarding stores a profile that drives all recommendations. Triage guides the care level. Matching shows a pre-filtered shortlist. Booking removes the phone-call drop-off. Prep and follow-up keep continuity. Prep and follow-up are largely future scope.

6. Process Architecture (APQC-style)

- Core process (value-creating):** Profile Setup, Triage, Doctor Match, Booking, Confirmation, Follow-Up. Every step is AI-executed. Humans act only as escalation exceptions.
- Management processes:** Doctor Network Management, Quality and Compliance Monitoring, AI Model Governance, Partner Insurance Integration, and Performance Reporting. *Demo reality:* public datasets were used (no insurance data, limited hours and

language). Doctor contacts are stripped. Prompts are engineered for scope. Analytics are manual via Supabase. Dashboards are future scope.

- **Support processes:** Customer Support (human escalation, chat-first), Doctor Scheduling API Maintenance, Language Model Updates, User Feedback Loop, and Clinic Onboarding. All are future integration.

7. BPMN 2.0: Designed vs Delivered

Legend: Live (✅), Partial (🟡), Future (🔵).

BPMN Element	Step	Owner	Status
Start Event	Open app / log in	System	✅ index.html Supabase Auth; new user picks language
User Task	Language & profile setup	User+System	🟡 Live but amanual form (homepage.html), not a conversational agent. Saves to profiles
Service Task	Insurance policy retrieval	AI	🔵 No insurer API;INSURANCE_FILTER_ENABLED=false scaffold
Gateway (XOR)	Referral required?	AI	🟡 referral_required flag + GP-referral footnote; auto GP pre-booking not built
Service Task	Symptom triage (navigation)	AI	✅ Groq via symptom-check (or n8n Triage Agent); care type, never diagnosis
Gateway (XOR)	Urgent care?	AI	✅ urgency routine/soon/emergency; emergency shows 112
Manual Task	High-stakes routing review	Human-in-loop	🟡 needs_human_review flag + n8n Send and Wait; reviewer not staffed
Service Task	Doctor matching	AI+System	✅ DB-first doctors query (postcode/city, language, specialty) + OSM fallback
User Task	Doctor selection	User	✅ one-tap (doctors.html)
Service Task	Appointment booking	AI+System	✅ writes appointments (Supabase or n8n); no AI voice call (ArztAPI scaffold off)
Gateway (XOR)	Booking successful?	System	✅ 0-row writes surfaced; closed-day/invalid slots rejected
Service Task	Confirmation & prep	AI+System	🟡 in-app confirmation from saved row; no booking email ; no-hours doctors get a mailto: to the user's own inbox
Intermediate Event	Day-before reminder	Timer	🔵 no scheduler
Service Task	Post-appointment follow-up	AI	🟡 reschedule/cancel live (audit logs); prescriptions/feedback future
End Event	Care cycle complete	System	✅ persisted; cancel/reschedule archived
Exception Flow	AI cannot resolve	Human	🔵 escalation not staffed

As built: of 16 elements, **9 are live, 4 are partial, and 3 are future.** Triage, matching, and booking are all AI/automated and connected end-to-end.

8. Regulatory, Compliance, and Governance

The product sits at the intersection of **GDPR** (data protection) and the **EU AI Act** (risk), with **MDR** (medical device) behind triage. The **key insight is that the regulatory burden is concentrated in one feature: the triage engine.** Matching and booking are comparatively low-risk.

Component	Regime	Risk	Burden
Doctor matching	EU AI Act (limited)	Low	Transparency labels
Automated booking	GDPR Art. 22	Low-Med	Consent + optional human override
Insurance explainer	GDPR	Medium	Consent, output accuracy
Symptom triage	EU AI Act (poss. high-risk) + MDR	High	Conformity assessment, human oversight
All health data	GDPR Art. 9 + DPIA	High	Explicit consent, DPIA, minimization

- **GDPR:** health data is special-category (Art. 9). The legal basis is explicit opt-in consent. A **DPIA is mandatory** before launch. Data minimization keeps only insurance type, postcode/city, and language. For Art. 22, low-stakes booking is automated and high-stakes routing goes through the human-in-the-loop safeguard.
- **EU AI Act:** triage is the high-risk question. The **mitigation is "navigation, not diagnosis."** It recommends a *type of care*, never a condition. This is a contested line, so it is paired with the human safeguard. Transparency is also kept: users are always told they interact with AI.
- **Compliance by design (governance roles):** DPO (DPIA, consent, breach), AI Governance Lead (risk classification), Clinical/Ops (human oversight), Engineering (audit and logging), Compliance Lead (horizon scanning).

Delivered vs future compliance controls:

Control	Status	In the build
"Navigation, not diagnosis"	✅	Enforced in Groq triage prompt

Control	Status	In the build
Emergency escalation (112)	✓	urgency:emergency + needs_human_review, UI guidance
AI Act audit log	✓	Every triage written to consultations
Data minimization	✓	Only insurance/postcode/city/language; contacts stripped
RLS / data isolation	✓	Per-user RLS; INSERT-only invisible audit tables
Explicit consent (Art. 9)	●	No opt-in screen yet
Human-in-the-loop reviewer (Art. 22)	●	Flag +Send and Wait; not staffed
DPIA / DPO / conformity	●	Documented as governance roles, not implemented

9. Interface (GUI/VUI), Inputs, and Outputs

The product is designed for two interaction modes. **Only the GUI is built** (iPhone-style frame, 6 languages). The **VUI** (speak and listen) is documented future scope. It is just an I/O wrapper around the same triage logic (Speech-to-Text, AI Agent, Text-to-Speech).

GUI screens to process steps: Welcome/Entry and Language (index.html). Profile Survey (homepage.html, a manual form). Symptom Input and AI Recommendation (symptom.html). Availability and Doctor List (doctors.html). Appointment Slot and Confirmation (booking.html). The slot picker is constrained to parsed opening hours. No-hours doctors fall back to an email draft.

I/O map: Entry writes auth/session. Profile writes profiles. Symptoms go to Groq and write consultations. Urgency routes to 112 or standard. Match queries doctors plus OSM and returns a list. Selection carries the doctor to booking. Booking writes appointments (plus the FK). Confirmation reads back in-app (no email). Reschedule/Cancel write the audit tables.

System outputs: on-screen recommendation, urgency, shortlist, and in-app confirmation are live. Stored data is profiles, consultations, appointments, plus the invisible audit logs. The **only outbound email is the signup confirmation** (Supabase Auth via Mailjet SMTP; an explicit Mailjet node in n8n FLOW 4). The booking confirmation email and the Outlook calendar event are future scope.

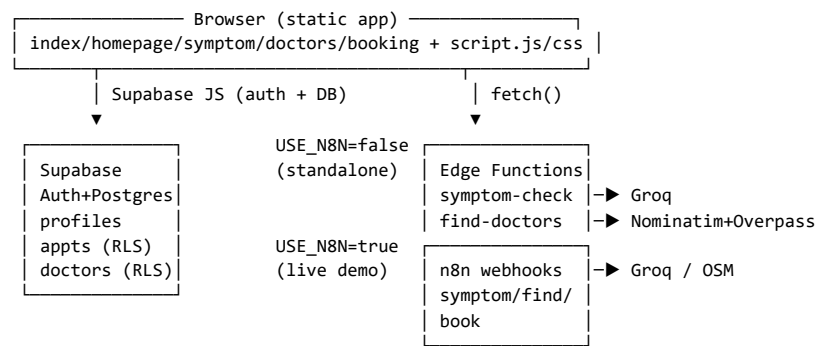
PART B. SYSTEM ARCHITECTURE AND BUILD

10. What It Is and the Stack

This is an AI-first patient-navigation prototype. The flow is symptoms, then a recommended doctor type and urgency, then find a doctor (a GP or an AI-recommended specialist), then book, reschedule, or cancel. The UI is multilingual (**English, Spanish, German, Turkish, Hindi, Arabic**) and styled as an iPhone-framed mobile app.

- Frontend:** static index/homepage/symptom/doctors/booking.html plus shared script.js and style.css. It is fully static, so it can be hosted on GitHub Pages. It talks to the live backend from the browser. The anon key is browser-safe and RLS protects the data.
- Backend:** Supabase. This is Auth plus Postgres (profiles, appointments, appointments_cancel, appointments_reschedule, doctors, consultations) plus Edge Functions (symptom-check, find-doctors).
- AI:** Groq LLM (llama-3.3-70b-versatile) via symptom-check. The key is server-side.
- Open data:** OpenStreetMap (Nominatim and Overpass) for the doctor fallback.
- n8n:** optional. The same three flows exist as webhooks for the live demo.

11. Architecture



12. Run Modes (one switch flips the whole app)

There are two flags at the top of script.js.

Flag	Default	Effect
USE_N8N	false	false means booking is direct-to-Supabase and symptom plus doctor-fallback use Edge Functions. true means all three route through n8n
N8N_PRODUCTION	true	true means n8n production webhooks (Active, hands-free). false means test webhooks (manual "Listen for test event")

Scenario	USE_N8N	N8N_PRODUCTION
Self-contained (own ecosystem)	false	true
Live n8n walkthrough (manual)	true	false
Hands-free through n8n	true	true

ARZT_API_ENABLED (ArztAPI), INSURANCE_FILTER_ENABLED, and DOCTOR_CALENDAR_ENABLED are all false scaffolds.

13. Pages and Flows

- **index.html:** register or log in (Supabase Auth). New users pick a language.
- **homepage.html:** the first visit completes the profile (insurance, postcode, city). A returning user sees a greeting plus three actions: **Find a Doctor** (GPs), **Find a Specialist Doctor** (the AI's recommendation only), and **Your appointments** (a bottom sheet). The avatar opens the Patient Information sheet (view/edit, sign out).
- **symptom.html:** describe symptoms. The AI returns an urgency badge, an explanation, a recommended specialty, and a referral note. "Find this doctor" carries the specialty to the search.
- **doctors.html:** Postcode and City (autofilled, editable) plus a language filter. Cards have Book, Call (valid phone only), and Website.
- **booking.html:** a doctor summary plus opening hours. If there are hours, the user books a date/time slot (limited to open days and times). If not, the user gets an email-request fallback. It also handles reschedule.

Doctor routing: the primary source is the imported doctors table. It is filtered by postcode/city, language, and specialty via `matchesSpecialty()`, which normalizes British/American spelling. If the DB returns nothing, it falls back to live OSM.

14. Data Model and Security (RLS)

This is the consolidated SQL for the Supabase SQL Editor.

doctors (public directory):

```
alter table public.doctors enable row level security;
create policy "Public can read doctors" on public.doctors for select using (true);
```

appointments (per-user, owner-only):

```
alter table public.appointments enable row level security;
create policy "owner can select appointments" on public.appointments for select using (auth.uid() = user_id);
create policy "owner can insert appointments" on public.appointments for insert with check (auth.uid() = user_id);
create policy "owner can update appointments" on public.appointments for update using (auth.uid() = user_id) with check (auth.uid() = user_id);
create policy "owner can delete appointments" on public.appointments for delete using (auth.uid() = user_id);
```

appointments_cancel and appointments_reschedule (write-once audit logs, INSERT-only, invisible to users):

```
alter table public.appointments_cancel enable row level security;
create policy "insert own cancellations" on public.appointments_cancel for insert with check (auth.uid() = user_id);
alter table public.appointments_reschedule enable row level security;
create policy "insert own reschedules" on public.appointments_reschedule for insert with check (auth.uid() = user_id);
```

consultations (each triage session; the AI audit log):

```
create table if not exists public.consultations (
  id bigint generated always as identity primary key,
  user_id uuid references auth.users(id) on delete cascade,
  symptoms text, recommended_doctor_type text, urgency text,
  referral_required boolean, needs_human_review boolean, language text,
  created_at timestamptz default now()
);
alter table public.consultations enable row level security;
create policy "insert own consultations" on public.consultations for insert with check (auth.uid() = user_id);
create policy "select own consultations" on public.consultations for select using (auth.uid() = user_id);
```

Key model notes:

- **Cancel is archive-before-delete.** `cancelAppointment()` copies the row to `appointments_cancel` before it deletes from `appointments`. Users have INSERT but no SELECT, so a cancellation never resurfaces. The table exists for metrics (cancel/reschedule frequency, booking-to-cancel lead time) and malicious-booking safeguards. Read it via the dashboard or `service_role`. The archive is best-effort. A failed archive never blocks the cancel. A source column ('`user_cancel`', future '`system_override`') marks the origin.
- **Reschedule is an UPDATE in place.** `handleBooking()` inserts one row to `appointments_reschedule`. It records `old_datetime`, `new_datetime`, `booked_at`, and `rescheduled_at`. It uses the same invisible, INSERT-only, best-effort pattern.
- **Wall-clock time.** `datetime` is stored as a naive string (YYYY-MM-DDTHH:MM:00) and read with `apptParts()`. It is never timezone-converted, so the displayed time always equals the booked time.

- **doctor_id FK.** The legacy `doctors_dummy` table is retired. `appointments.doctor_id` is now a real FK to `doctors(id)` (on delete set null). It is populated for DB doctors and null for OSM-fallback doctors. The `doctor_name`, `doctor_specialty`, and `doctor_address` text snapshot still records who the appointment is with.
- **Address behavior (intentional).** DB doctors show postcode and city only, because the scraped `street/address` is unreliable. OSM doctors carry a clean full street. This is a data-quality choice, not a bug.

15. Feature Areas

- **Appointments:** inline counts, then a button, then a bottom sheet (Reschedule / Cancel). Cancel uses a styled in-app modal, not `window.confirm`.
- **Booking availability:** hours are parsed from the 7 day columns (`hours_mon` to `hours_sun`). The date is restricted to open weekdays and the times to that day's ranges. Closed-day attempts are rejected.
- **Doctor search:** DB-first with OSM fallback. Matching is postcode-first, because the city field is typo-prone. The language filter defaults to the user's preferred language.
- **Email fallback safety:** "Request appointment by email" opens a draft to the user's own inbox, never a real practice.
- **Phone guard:** the Call button only shows when the phone has 6 or more digits, because the dataset has junk values.
- **Inactive scaffolds:** `ArztAPI` (`ARZT_API_ENABLED=false`), insurance filter (`INSURANCE_FILTER_ENABLED=false`), and doctor calendar sync (`DOCTOR_CALENDAR_ENABLED=false`).

16. Internationalization and UI Frame

- There are six languages with full key parity, verified programmatically. French was replaced by Turkish. The old in-app translation feature (translate-to-German notes, German doctor-summary) was removed.
- The app uses an iPhone-style black bezel (`.phone`) with an inner scrolling screen (`.screen`). The scrollbar is hidden. There is a floating back-to-top button. Bottom sheets and the confirm modal are anchored inside the frame.
- **Fixed phone shape on desktop:** the frame is locked to a 390:844 shape via CSS `aspect-ratio`. It scales down evenly on short or zoomed screens. **Real mobile view:** a media query at 480px or less drops the bezel and fills the screen using `100dvh`.

17. Build Log (chronological)

1. Appointments access (navigable overview plus sheet).
2. Reschedule safety (match by `id`, `UPDATE` in place, RLS, surface 0-row writes).
3. Wall-clock time fix.
4. Confirmation reads the persisted row.
5. Per-note copy buttons, then the translation feature was removed.
6. `ArztAPI` scaffold.
7. Homepage rebrand to ClearCare+.
8. Doctor DB-first routing plus OSM fallback; languages un-paywalled.
9. City field; Find-a-Doctor is GP and Specialist is the AI rec.
10. UX work (back-to-top, slim scrollbar, bezel, cancel modal).
11. Languages re-translated plus Turkish, full parity.
12. Insurance filter, opening-hours restriction, calendar scaffold, email-request fallback.
13. Doctor data: parsed `doctors_raw.xlsx` to `doctors.csv`, then import plus RLS.
14. Standalone vs n8n flags.
15. Edge Functions (symptom-check, find-doctors); fixed CORS and Overpass reliability.
16. Responsive phone frame (`aspect-ratio` plus 480px mobile view).

18. File Map

Path	Purpose
<code>index/homepage/symptom/doctors/booking.html</code>	App pages
<code>script.js</code>	All logic (router, auth, i18n, search, booking, scaffolds, run-mode flags)
<code>style.css</code>	Styles plus device frame
<code>supabase/functions/symptom-check/index.ts</code>	Groq proxy (AI triage)
<code>supabase/functions/find-doctors/index.ts</code>	OSM doctor fallback
<code>doctors.csv</code> / <code>doctors_raw.xlsx</code>	Cleaned import file / original source

PART C. EDGE FUNCTIONS (Standalone Engine)

Two Edge Functions let the app run entirely inside Supabase with no n8n. They are used when `USE_N8N=false`. Both return CORS headers (`Access-Control-Allow-Origin: *`).

Function	Code	Purpose	Secret
<code>symptom-check</code>	<code>supabase/functions/symptom-check/index.ts</code>	AI triage via Groq (key server-side)	<code>GROQ_API_KEY</code>
<code>find-doctors</code>	<code>supabase/functions/find-doctors/index.ts</code>	OSM fallback (Nominatim and Overpass) for cities not in the DB	none

`find-doctors` needs no secret, but it **must** be server-side. Nominatim requires a `User-Agent` that browsers cannot set, and the server side avoids CORS.

Deploy with the Dashboard (no CLI): for each function, go to Edge Functions, then Create a function, then name it exactly, then paste `index.ts`, then Deploy, then disable "Verify JWT". These are public browser endpoints. Then add `GROQ_API_KEY = gsk_...` under Edge Functions, then Manage secrets. Only `symptom-check` needs it.

Deploy with the CLI:

```
npm i -g supabase
supabase login
supabase link --project-ref ossxctklhsoiigixvvt
supabase secrets set GROQ_API_KEY=gsk_XXXXXXX
supabase functions deploy symptom-check --no-verify-jwt
supabase functions deploy find-doctors --no-verify-jwt
```

Test:

```
curl -i -X POST "https://ossxctklhsoiigixvvt.supabase.co/functions/v1/symptom-check" \
-H "Content-Type: application/json" -H "apikey: <anon key>" \
-d '{"symptoms":"sore throat and mild fever for 3 days","language":"English"}'
# returns { "urgency":"routine", "explanation_for_user":"...", "recommended_doctor_type":"General Practitioner", "referral_required":f
```

Notes: find-doctors tries several Overpass mirrors with timeouts. If all fail, it returns 200 with [] (the UI shows "no doctors found") and logs the reason. The model is llama-3.3-70b-versatile. Change MODEL if Groq deprecates it. The Groq key lives only in the Supabase secret, never in the browser.

PART D. n8n WORKFLOW (Demo Engine)

19. Why n8n, and the Auth Boundary

n8n mirrors the **back-end process**. It does **not** own login. Supabase Auth owns login. n8n works on the **data tables** plus the **AI triage**, which is exactly what the BPMN automates. There are two doors. The **Auth API (GoTrue)** is used by the app's supabase-js for sign-up, login, and auth.users. The **Database API (PostgREST)** is used by n8n's Supabase node. n8n gets the **service_role (secret) key** in its credential. This stays in n8n only, never the browser. It bypasses RLS for backend inserts.

20. The One Consolidated Workflow

Import the single file `Audit/Documentation/Workflow JSON SQL/ClearCare+ Complete Workflow.json`. You get one canvas with all the webhook entry points. There are 26 nodes, no duplicate node names, and the register/login flow is removed.

Reviewer note. The live n8n link on the cover (`pcsphsb.app.n8n.cloud/...`) is the team's private n8n Cloud workspace. It opens only for an account with access, so it cannot be viewed by clicking the link alone. To inspect or run the workflow yourself, import the JSON file above into your own n8n account (free Cloud or self-hosted): Workflows, then Import from File. Re-link the Supabase and Groq credentials when prompted (see Section 24).

Webhook path	Flow on canvas
symptom-check	Webhook, Load Profile, Medical Triage Agent (Groq + Structured Output Parser), Code (parse), Respond, Route by Urgency, Emergency or Save Consultation, Find Matching Doctors, Merge, Respond Success
find-doctors	Webhook, Geocode (Nominatim), Overpass, Normalize Doctors, Respond
book-appointment	Webhook, Create Appointment (Supabase), Respond
FLOW 4 (signup confirmation email)	Supabase Admin: Generate Link, Build Confirmation Email, Send via Mailjet, Respond

"Simultaneous" clarified: n8n allows several Webhook triggers on one canvas, but they are **not one execution**. The app calls them at three separate moments, which is three HTTP calls. For the demo, use two screens: the app UI and this n8n tab. As you click through, the Executions panel lights up each flow. A single linear run would need a fake orchestrator and would misrepresent the app.

21. Cleanup Performed

- Removed the deprecated register/login flow. It wrote profiles via a table hook, not Supabase Auth, so it is inaccessible in the current app.
- Rebranded every MediCare+ to ClearCare+ (workflow, node, and sticky names, plus the source JSON).
- Set CORS * on every Webhook node.
- Archived the obsolete translate-note workflow to Deprecated/. The translation feature was removed.
- Removed the old booking/welcome email nodes. The booking confirmation is shown in-app from the saved row. No booking email is sent.

22. Registration Email (FLOW 4)

The app **does** send one email: the **signup confirmation**. It goes through Supabase Auth using **Mailjet as the custom SMTP relay**. This is a dashboard setting, with no Mailjet code in the app. FLOW 4 mirrors this for the demo and makes Mailjet a visible node. The steps are Supabase Admin: Generate Link (creates the user and returns the confirmation link without auto-sending), then Build Confirmation Email

(injects the link into the branded template), then Send via Mailjet (Send API v3.1). generate_link is used instead of /auth/v1/signup so the explicit Mailjet node does not double-send. It requires the Supabase **service-role** key (server-side) plus a Mailjet Basic Auth credential. The sender is ClearCare+ <clearcareplus.de@gmail.com>.

23. NULL-Field Fixes (main correction)

Two Supabase insert nodes wrote NULLs from wrong expressions.

- 1. **Save Consultation** read {{ \$("Medical Triage Agent").item.json.output.* }}, which was undefined because the Agent returns the JSON as text. It was fixed to read the **Code node** that parses it: {{ \$("Code in JavaScript").item.json.recommended_doctor_type }} (plus urgency, referral_required, needs_human_review). symptoms and user_id come from the webhook body. language comes from the loaded profile.
- 2. **Create Appointment** originally mapped only doctor_id and broke on a non-existent value. It now maps doctor_name, doctor_specialty, doctor_address, datetime, user_id, and doctor_id. doctor_id is populated for DB doctors and null for OSM doctors. This matches the app payload and the live appointments table, which stores the text snapshot and the doctor_id FK.

24. Import, Activate, Present

- 1. In n8n, go to Workflows, then Import from File, then ClearCare+ Complete Workflow.json.
- 2. Re-link credentials if prompted (the Supabase and Groq nodes). They are mapped by name if they exist.
- 3. Confirm each Webhook node has CORS * and Respond set to "Using Respond to Webhook node".
- 4. **Activate** for production URLs and hands-free, or leave it inactive and use "Listen for test event" for a manual step-by-step demo.
- 5. In script.js, set USE_N8N=true (and N8N_PRODUCTION=false for the manual walkthrough). Set it back to false for the standalone engine.

Status: the live n8n runs on cloud n8n (pcsphsb.app.n8n.cloud), not the older localhost:5678 examples. The Symptom Checker, doctor search, and booking are all built and live.

25. AI Triage System Prompt (Groq Medical Triage Agent)

The prompt is framed as **navigation, not diagnosis** (see compliance, Section 8). It is about 1,500 characters; the n8n prompt limit is 5,000. The output is strict JSON. Downstream, a **Code node** strips ```json fences, because Groq plus the Structured Output Parser is unreliable, and it falls back to key:value parsing.

You are the triage assistant for ClearCare+, a health-insurance navigation app for immigrants in Germany. You help users reach the RIGHT TYPE of doctor. You are a navigator, NOT a diagnostician.

RULES:

- 1. Recommend ONE doctor type only (GP, Dermatologist, Cardiologist, ENT, Orthopedist, Gynecologist, Pediatrician, Psychiatrist, Opht
- 2. NEVER name a disease, diagnosis, or treatment.
- 3. "urgency" is EXACTLY one of: routine, soon, emergency.
- 4. EMERGENCY OVERRIDE: red flags (chest pain, difficulty breathing, stroke signs, severe bleeding, fainting, suicidal thoughts, seve
- 5. GERMAN REFERRAL RULE: specialists need a GP referral, so referral_required=true for a specialist and false for GP/Emergency.
- 6. Write "explanation_for_user" in the user's language: {{ \$json.language }}. Warm, simple, 1-2 sentences.
- 7. If symptoms are vague, default to GP. Prefer "soon" over "routine" if painful, worsening, or persistent.

Return ONLY valid JSON:

```
{"recommended_doctor_type":"string","urgency":"routine|soon|emergency","referral_required":true,"needs_human_review":false,"explanat
```

Reference the webhook explicitly in the User Message, because it sits after the "get profile" node: Symptoms: "{{ \$('Webhook').item.json.body.symptoms }}" , Language: {{ \$('Webhook').item.json.body.language }}. Downstream nodes then read clean top-level fields ({{ \$json.urgency }}, and so on).

26. n8n Workflow Files

File	Role
ClearCare+ Complete Workflow.json	Import this. The consolidated 3-flow (plus FLOW 4) workflow
find-doctors-workflow.json	source export of the OSM flow (reference)
book-appointment-workflow.json	older standalone booking export (reference)
SQL/...	table/RLS setup (see Section 14 for the verified schema)
Deprecated/translate-note-workflow.json	archived; the translation feature was removed

PART E. DOCTOR DATA

27. Source and Import

- **Source:** doctors_raw.xlsx (1,174 rows, untouched). **Import file:** doctors.csv (the same data with clean lowercase snake_case headers). Regenerate it from the xlsx with a Node converter. The converter renames headers only and copies rows verbatim. It unzips the xlsx, reads sharedStrings.xml plus sheet1.xml, and writes the CSV.
- **App side:** findDoctorsFromDB() queries doctors (primary) with an OSM fallback.

Columns and quality:

Header	Meaning	Quality
name	Doctor/practice	reliable
specialty	e.g. "General Practitioner / Family Medicine", "Gynaecology"	reliable; British spelling
address	Street	🚩 messy (~47% are scrape dates); not used for matching
postcode	5-digit	✅ 100% clean; primary location filter
city	City	🚩 typos ("Stuttgart"); fallback only
languages	Comma list	reliable; language filter
email	Contact	~56% present; email-request booking
phone	Phone	❌ ~73% junk; Call hidden unless 6+ digits
website	URL	~55% present
hours_mon to hours_sun	Per-weekday hours	731/1174 have usable hours

Import paths. Path A (easiest): Table Editor, then New table, then "Import data from CSV", then upload `doctors.csv`. It creates 16 columns plus `id` automatically. Name it `doctors`, then apply RLS. **Path B (explicit types):** run the CREATE TABLE first, then Insert, then Import data from CSV. A header-mismatch error means you imported into a table that lacks those columns. Use Path A, or run `drop table if exists public.doctors;` and retry.

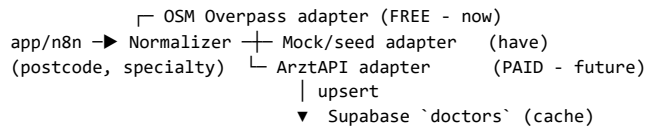
```
create table public.doctors (  
  id bigint generated by default as identity primary key,  
  name text, specialty text, address text,  
  postcode text, city text, languages text,  
  email text, phone text, website text,  
  hours_mon text, hours_tue text, hours_wed text, hours_thu text,  
  hours_fri text, hours_sat text, hours_sun text  
);  
create index doctors_postcode_idx on public.doctors (postcode);  
create index doctors_city_idx on public.doctors (city);  
create index doctors_specialty_idx on public.doctors (specialty);
```

RLS is read-only-for-everyone (see Section 14). If a search returns nothing despite rows existing, the SELECT policy is almost always missing.

How the app uses it: location is filtered by `postcode` (exact; `city` is used only when no `postcode` is given). Specialty uses `matchesSpecialty()`, which normalizes "Gynecologist" and "Gynaecology". Languages use `ilike` on the comma list. Hours come from the 7 columns. The phone is validated before "Call" shows. **Future columns (scaffolded):** `accepts_insurance` (flip `INSURANCE_FILTER_ENABLED=true`) and doctor calendar sync (`DOCTOR_CALENDAR_ENABLED`).

28. Provider Adapter Pattern (free now, ArztAPI-pluggable later)

Problem: there is no open German doctor DB (GDPR), and the real product, **ArztAPI**, costs about €799/month. **Goal:** a pluggable architecture that runs on free data now and lets anyone drop in ArztAPI later without a rewrite.



The normalizer plus cache stays constant forever. Swapping to ArztAPI means adding one adapter and flipping a `PROVIDER` variable. A Set node feeds a Switch node that routes to the matching adapter, and all adapters feed the same normalized output. The **normalized contract** mirrors the ArztAPI fields, so a swap is 1:1: `name`, `specialty`, `languages`, `insurance_accepted`, `street`, `postcode`, `city`, `lat/lon`, `phone`, `website`, `source`, `osm_id`.

The **live** `doctors` table differs from the aspirational contract. It uses a single `address` column (not `street` plus `lat/lon`), `accepts_insurance` (not `insurance_accepted`), 7 `hours_*` columns, and no `osm_id` or `source`. Treat Section 28 as the design contract and Sections 14 and 27 as what is actually running.

OSM proof (overpass-turbo.eu), city version:

```
[out:json][timeout:25];  
{{geocodeArea:Bremen, Germany}}->.a;  
( node["amenity"="doctors"](area.a); node["healthcare"="doctor"](area.a); );  
out body 60;
```

Coordinate/around version (works in the raw API and n8n; the {{geocodeArea}} macro is overpass-turbo only):

```
[out:json][timeout:25];
( node["amenity"]="doctors")(around:6000,53.0793,8.8017);
  node["healthcare"]="doctor"(around:6000,53.0793,8.8017); );
out body 60;
```

n8n OSM pipeline: Webhook (find-doctors, CORS *), then Nominatim geocode (postcode to lat/lon, User-Agent required), then the Overpass around query (the data param), then the Code normalizer, then Supabase upsert (match osm_id, which caches), then Respond.

Normalizer (n8n Code node, Run Once for All Items):

```
const els = $json.elements || [];
const SPECIALTY_MAP = { general:"General Practitioner", dermatology:"Dermatologist",
  cardiology:"Cardiologist", paediatrics:"Pediatrician", gynaecology:"Gynecologist",
  orthopaedics:"Orthopedist", psychiatry:"Psychiatrist", ophthalmology:"Ophthalmologist",
  otolaryngology:"ENT" };
return els.filter(e => e.tags && e.tags.name).map(e => {
  const t = e.tags;
  const spec = (t["healthcare:speciality"] || "").split(";")[0].trim().toLowerCase();
  return { json: {
    osm_id: "osm-" + e.id, name: t.name,
    specialty: SPECIALTY_MAP[spec] || (spec || "General Practitioner"),
    languages: t["language"] || (t["language:en"] === "yes" ? "English" : ""),
    street: [t["addr:street"], t["addr:housenumber"]].filter(Boolean).join(" "),
    postcode: t["addr:postcode"] || "", city: t["addr:city"] || "",
    lat: e.lat, lon: e.lon,
    phone: t.phone || t["contact:phone"] || "",
    website: t.website || t["contact:website"] || "", source: "OSM"
  }};
});
```

Honest limitations (a strength to state): OSM coverage is incomplete. specialty is often untagged, so it falls back to GP. There is **no insurance (GKV/PKV) data**. There is no real availability. ArztAPI fills each gap with a complete registry, reliable specialty, insurance acceptance, and real-time slots. The free build is a best-effort directory that proves the architecture. The paid layer is data completeness, swapped in cleanly. **Fair use:** Nominatim allows about 1 request per second and requires a real User-Agent; do not bulk-scrape. The Overpass public instance is fair-use; cache results in Supabase.

PART F. ArztAPI SCAFFOLDING (inactive future integration)

ArztAPI is a **placeholder** for a future direct doctor-booking integration. It is **NOT live**. The n8n webhook (or direct Supabase) is the real active path. While ARZT_API_ENABLED=false, the ArztAPI branch is **dead code**. It makes no network calls and produces no errors. Its job is to document exactly where a real ArztAPI slots in, so future work is a seam and not a rewrite. It uses the **feature-flag plus adapter** pattern.

In script.js (fenced with START/END banners):

Symbol	Purpose
ARZT_API_ENABLED	Master switch (keep false)
ARZT_API_URL / ARZT_API_KEY	Future endpoint / auth (empty now)
bookViaArztApi(payload)	Adapter for new bookings; returns a mock while disabled (the real fetch is commented out)
rescheduleViaArztApi(payload)	Adapter for reschedules; same disabled-stub contract

handleBooking() has two branches, and each has a labeled fork. The **new booking** branch (rescheduleId==null) goes to ArztAPI (inactive) if ARZT_API_ENABLED, else to the **n8n webhook** (active). The **reschedule** branch (rescheduleId!=null) goes to ArztAPI (inactive), else to the **Supabase UPDATE** (active). A shared payload feeds both pathways.

Data shapes:

```
// Booking payload (sent to n8n today; for ArztAPI later)
{ "user_id":"uuid", "doctor_name":"string", "doctor_specialty":"string",
  "doctor_address":"string", "datetime":"YYYY-MM-DDTHH:MM:00" }
// Reschedule payload
{ "appointment_id":"id", "user_id":"uuid", "datetime":"YYYY-MM-DDTHH:MM:00" }
// Expected ArztAPI response (mocked today)
{ "ok":true, "source":"stub", "confirmation_id":"DUMMY-<timestamp>" }
```

Going live (checklist): 1. Set ARZT_API_ENABLED=true. 2. Fill ARZT_API_URL and ARZT_API_KEY. 3. Uncomment the real fetch in both adapters. 4. **Keep the read model in sync.** The UI reads from Supabase appointments, so the ArztAPI branch must still mirror the appointment to Supabase. The NOTE: comments mark where. 5. Test new booking and reschedule end-to-end. Confirm the times match. Preserve the yyyy-MM-DDTHH:MM:00 wall-clock format via apptParts().

PART G. FUTURE SPRINTS

- **Smart AI emergency routing.** This needs more precise routing prompts.
 - **Insurance-filtered results.** This needs a better DB or a dedicated API.
 - **End-to-end user-to-doctor communication.** This needs an API to the doctors' calendars plus stricter legal architecture for sensitive medical data.
 - **UI improvements.** Examples are prescription recording and a calendar widget that highlights marked dates.
 - **Mobile to web expansion.** The app is mobile-first now to avoid overengineering and empty-space filler.
 - **Legal architecture.** The Terms and Privacy links are non-functional placeholders right now.
 - **Cybersecurity layer.** Examples are 2FA, a stricter password policy (12 characters minimum, with numbers and symbols), and anti-bot captcha. This needs proper hosting, which is out of scope for a demo.
-

MDT 25-27 | Change and Process Management. ClearCare+ consolidated documentation. Long-form sources are README.md and Audit/Documentation/Workflow Documentation/. Workflow exports and SQL are in Audit/Documentation/Workflow JSON SQL/.